Sistemas Operacionais

Threads e escalomento.

Rafael Descio, M.Sc.

Católica SC
Centro Universitário

Objetivos da Aula de Hoje

- **Compreender** o conceito de thread, diferenciando-o de processo e reconhecendo seu papel como unidade de execução leve dentro do mesmo espaço de memória.
- **Analisar** o funcionamento de múltiplas threads em um processo, identificando compartilhamento de recursos, execução concorrente e os riscos associados, como condições de corrida.
- **Entender** o conceito de escalonamento de CPU, reconhecendo o papel do escalonador na seleção de threads prontas para execução e os estados fundamentais (ready, running, blocked).
- **Aplicar** conceitos de threads e escalonamento por meio da execução prática em C com POSIX Threads (pthread), observando na prática a concorrência, a não determinismo da execução e a atuação do escalonador.

O que são Threads?

Segundo Tanenbaum, o estudo de threads aparece dentro do capítulo de Processes and Threads de Modern Operating Systems, onde ele apresenta threads como uma forma de múltiplas linhas de execução existirem dentro do mesmo processo. Nas edições amplamente usadas da obra, o tópico de threads inclui uso, modelo clássico, POSIX Threads, implementação em espaço do usuário, no kernel e modelos híbridos.

O que são Threads?

Uma thread é uma unidade de execução dentro de um processo.

Em outras palavras:

- o processo é o contêiner de recursos;
- a thread é o fluxo de execução.

Um processo pode ter:

- **uma única thread**, caso mais simples;
- **várias threads**, executando tarefas diferentes dentro do **mesmo espaço de memória**.

Processo x Thread

Processo

Um processo possui, em geral:

- espaço de endereçamento próprio;
- variáveis globais;
- arquivos abertos;
- recursos do sistema;
- proteção e isolamento em relação a outros processos.

Thread

As threads de um mesmo processo compartilham:

- mesmo espaço de memória;
- mesmas variáveis globais;
- mesmos arquivos abertos;
- mesmos recursos do processo.

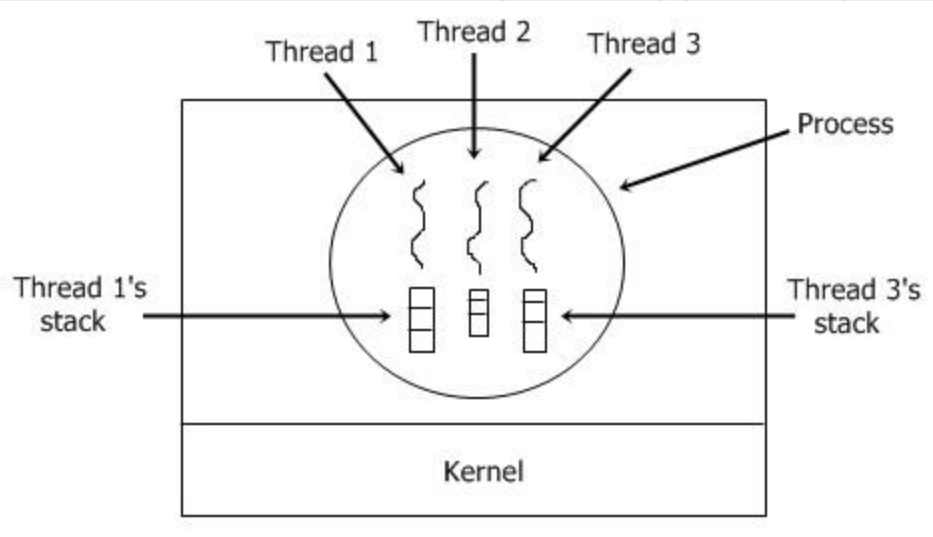
Processo x Thread

Mas cada thread possui seu próprio:

- contador de programa;
- registradores;
- pilha.

Esse é exatamente o ponto central do modelo clássico de threads: threads compartilham o contexto do processo, mas cada uma mantém seu estado de execução individual.

Clássico modelo de Thread



Threads em espaço do usuário e no kernel

Threads em espaço usuário (ULT)

A biblioteca de threads gerencia tudo sem que o kernel saiba de cada thread.

Vantagens

- criação mais rápida;
- troca de contexto mais rápida;
- independência do suporte do sistema operacional.

Desvantagens

- se uma thread faz uma chamada bloqueante, pode bloquear o processo inteiro;
- o kernel enxerga o processo, não cada thread individualmente.

Threads em espaço do kernel (KLT)

O sistema operacional conhece e gerencia cada thread.

Vantagens

- melhor escalonamento real;
- bloqueio de uma thread não precisa parar as outras;
- melhor uso de multiprocessadores.

Desvantagens

- maior overhead que threads puramente em usuário

Modelo híbrido

Há também o multiplexamento de threads de usuário sobre threads do kernel, combinando flexibilidade e escalabilidade.

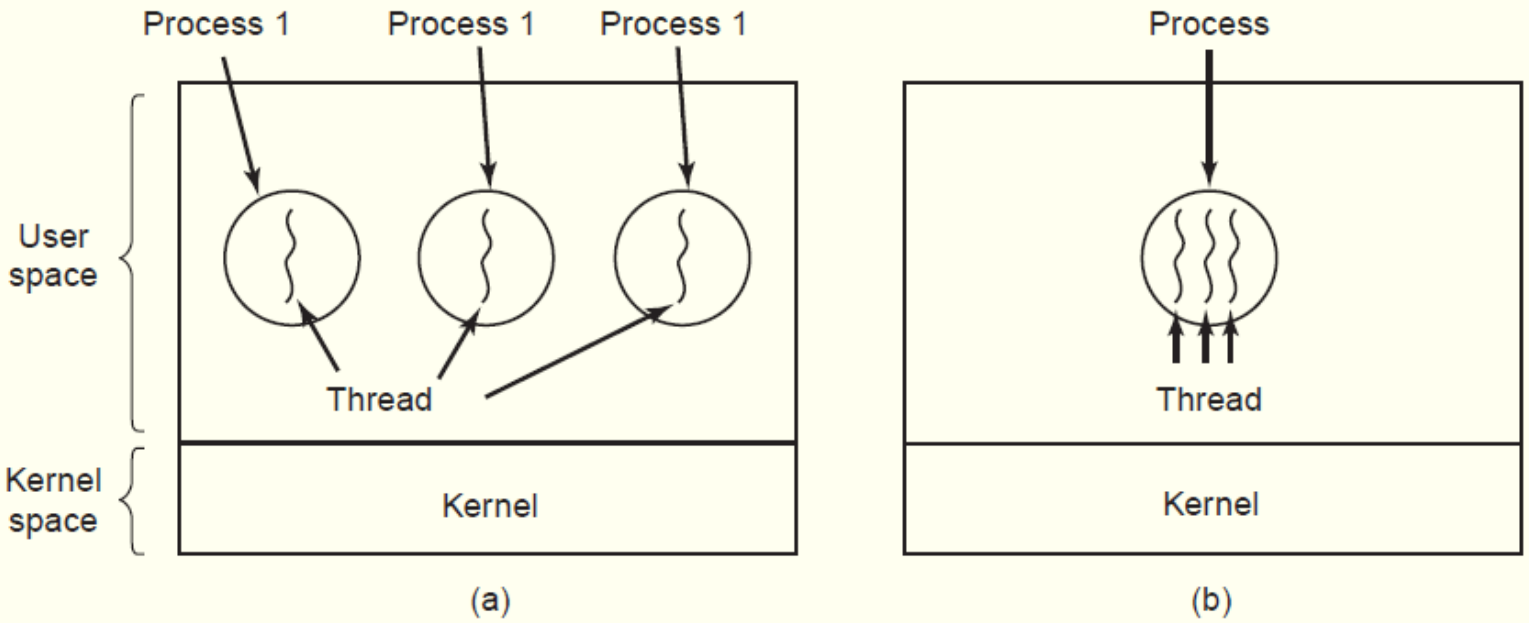
Vantagens

- Escalabilidade real (multi-core)
- Flexibilidade de agendamento
- Melhor desempenho em I/OS
- Redução de overhead
- Controle fino pela aplicação

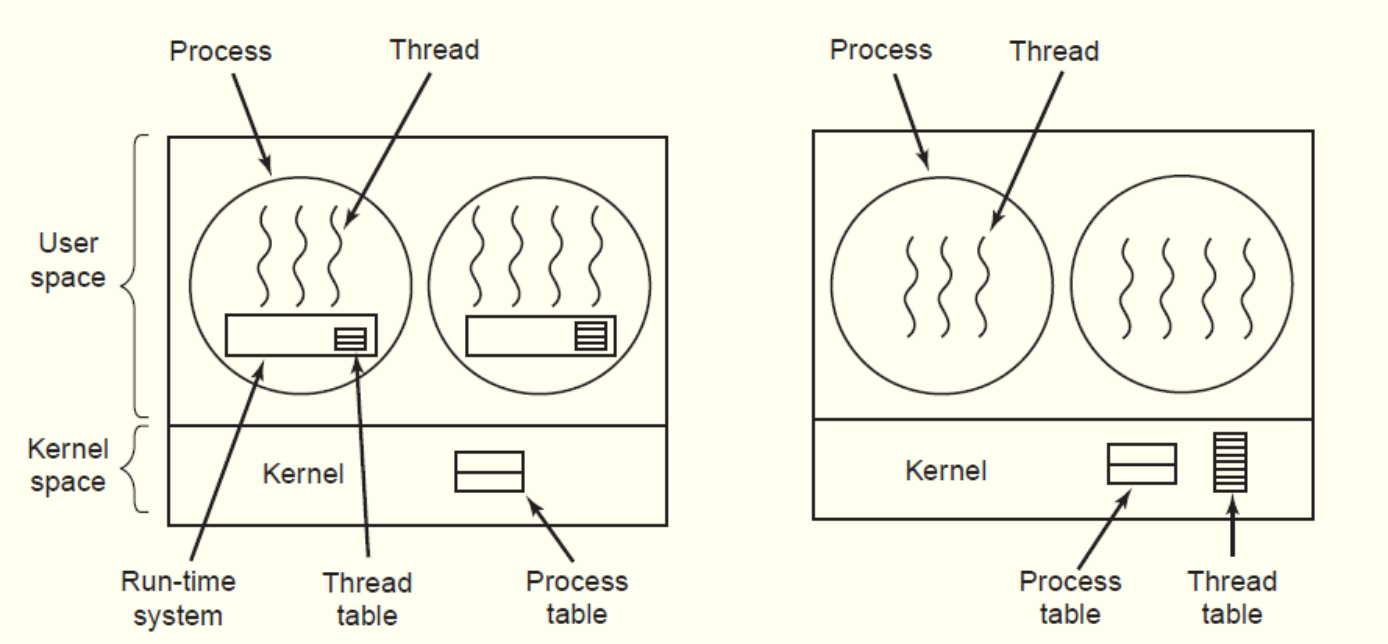
Desvantagens

- Alta complexidade de implementação
- Sincronização mais complexa
- Problemas com bloqueio de sistema
- Pouco suporte moderno
- Debug e profiling difíceis

Escalonamento de Threads

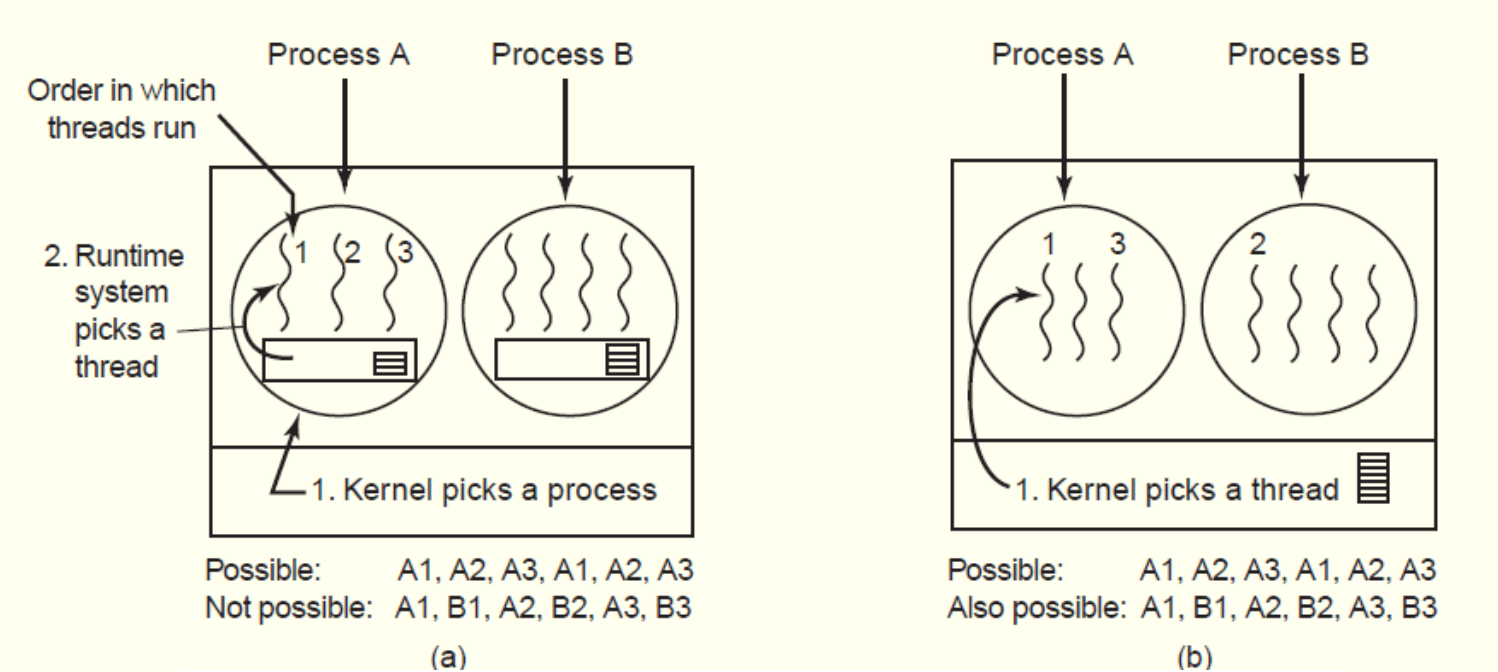


Escalonamento - Threads de usuário e de kernel



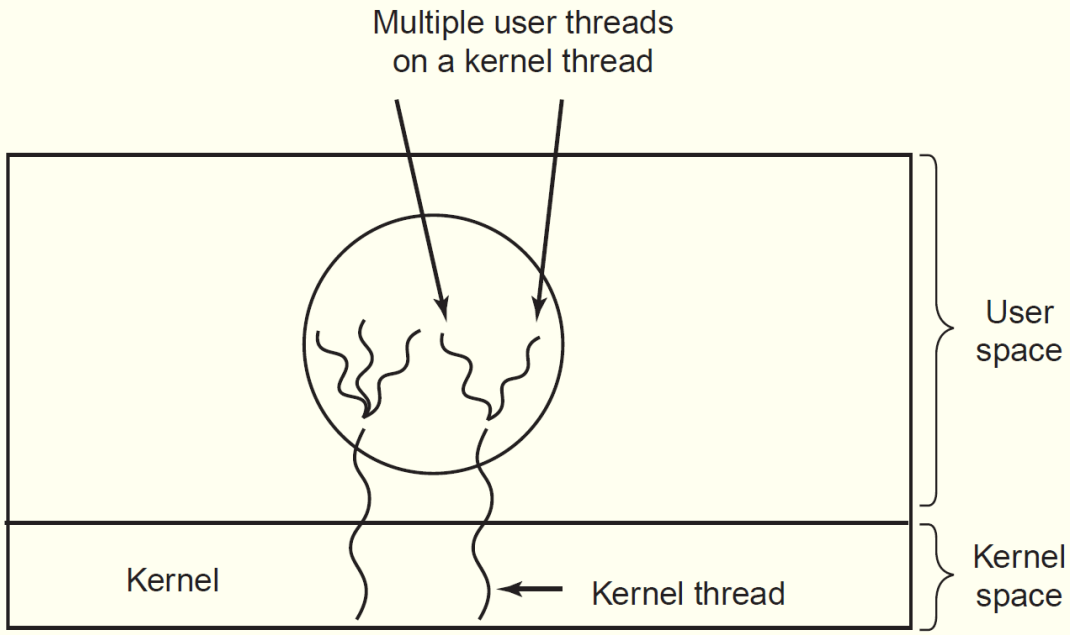
Threads ao nível de usuário vs Threads ao nível de kernel

Escalonamento - Threads de usuário e de kernel



Threads ao nível de usuário vs Threads ao nível de kernel

Escalonamento – Modelo Híbrido



Multiplexação de threads de nível de usuário e threads de nível de kernel.

Modelos de Threads: N:1 vs 1:1 vs M:N

Modelo	Descrição	Mapeamento
N:1	Várias threads de usuário em uma única thread de kernel	N → 1
1:1	Cada thread de usuário mapeia para uma thread de kernel	1 → 1
M:N	Várias threads de usuário multiplexadas em várias threads de kernel	M → N

N:1 — User-Level Threads

Conceito
- Threads gerenciadas por biblioteca em user-space
- Kernel não tem conhecimento das threads

Vantagens
- Alta performance (sem syscall)
- Troca de contexto leve

Desvantagens
- Sem paralelismo real (1 core)
- Bloqueio global (uma trava tudo)

Exemplo:
- Green threads (Runtime ou VM)
- Runtime de linguagens antigas

1:1 — Kernel-Level Threads

Conceito
- Cada thread é gerenciada pelo kernel

Vantagens
- Paralelismo real (multi-core)
- Bloqueio isolado por thread
- Melhor escalonamento

Desvantagens
- Overhead maior (systemcall)
- Criação mais cara

Exemplo:
- pthread (Linux)
- Windows threads

M:N — Modelo Híbrido

Conceito

- Muitas threads de usuário distribuídas em um pool de threads do kernel

Vantagens

- Combina desempenho + paralelismo
- Mais flexível

Desvantagens

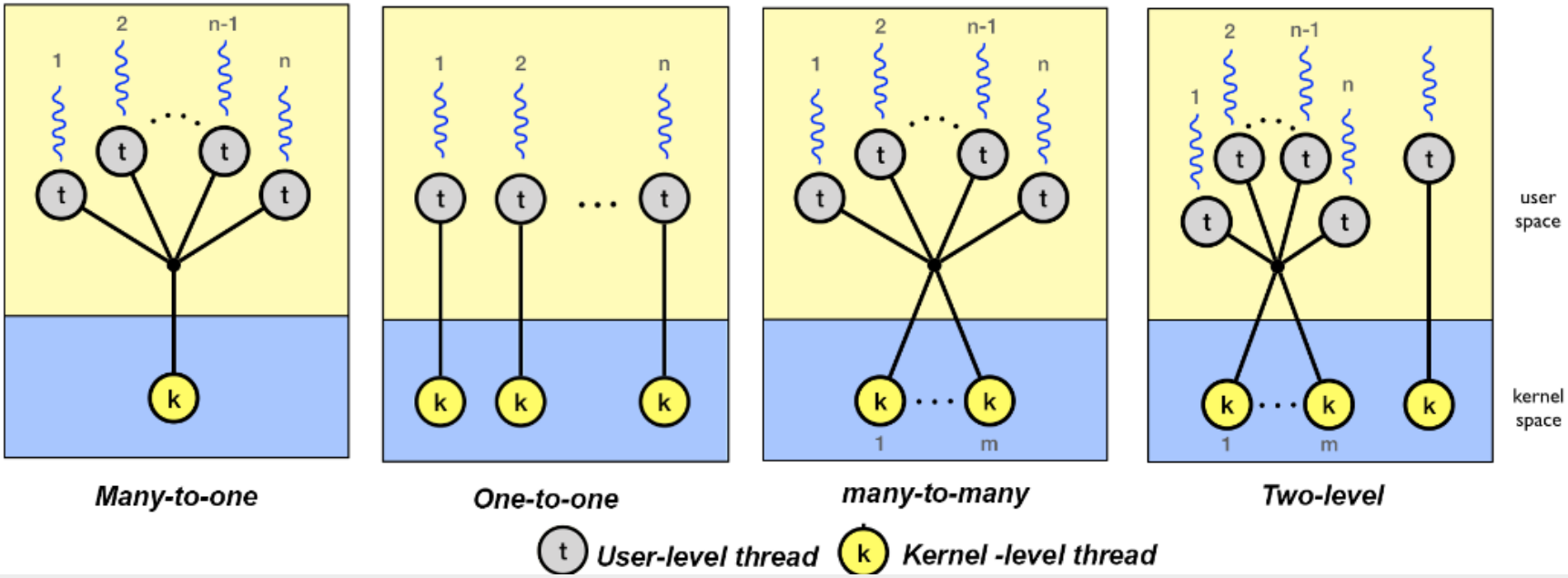
- Implementação complexa
- Pouco usado hoje em dia

Exemplo:

- Solaris (antigo)
- Go (goroutines – mesmo conceito)

Modelos de Threads em Nível de Usuário

User-level thread models



Por que usar Threads?

O uso de threads tem como principal função melhorar estruturação e desempenho de aplicações, especialmente quando há atividades simultâneas ou sobreposição entre processamento e espera por I/O.

Vantagens principais

a) Responsividade

Uma parte do programa pode continuar ativa enquanto outra espera.

Exemplo:

- interface gráfica continua respondendo;
- enquanto isso, outra thread salva arquivo ou busca dados.

Por que usar Threads?

Vantagens principais

b) Compartilhamento simples de dados

Como as threads compartilham memória, a troca de informações é mais rápida que entre processos.

c) Menor custo que processos

Criar e alternar threads costuma ser mais leve do que criar e alternar processos.

d) Paralelismo e concorrência

Em máquinas multicore, threads podem realmente executar em paralelo.

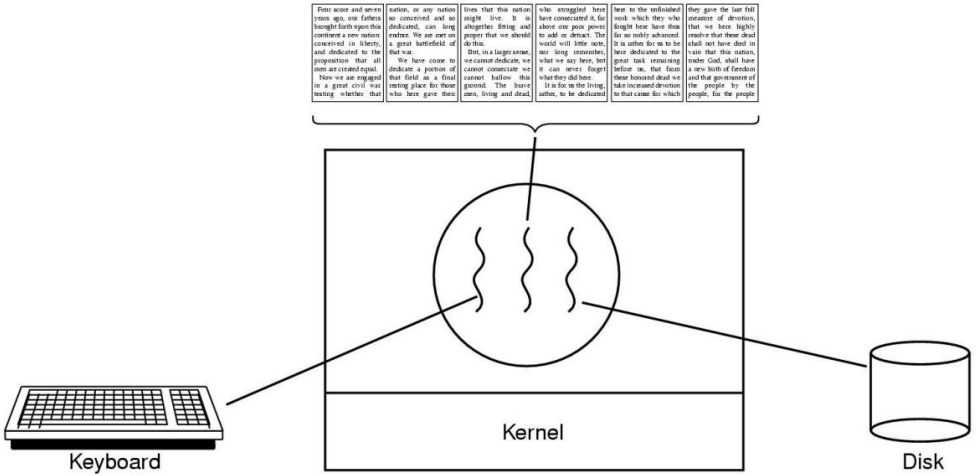
Exemplo

Imagine um navegador web:

- uma thread recebe cliques do usuário;
- outra baixa imagens;
- outra renderiza a página;
- outra toca vídeo ou áudio.

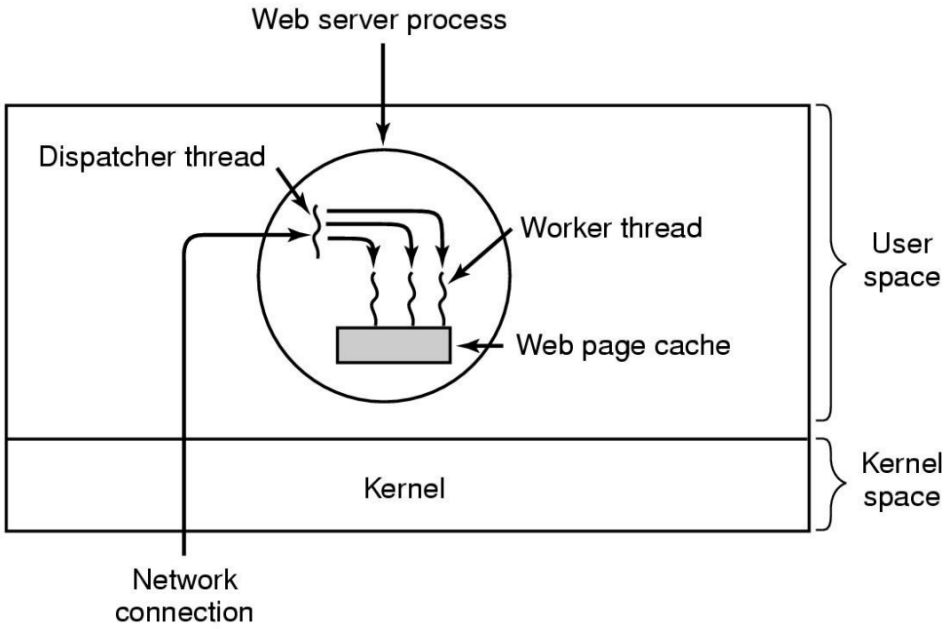
Tudo isso pode ocorrer dentro do mesmo processo, com várias threads cooperando.

Exemplo



Um processador de texto com três threads

Exemplo



Um servidor Web multithread

Condição de corrida (Race condition)

O que é Race Condition?

- Ocorre quando múltiplas threads/processos acessam dados compartilhados
- O resultado final depende da ordem de execução (timing)
- Essa ordem é não determinística

Definição (Tanenbaum):

- Situação em que múltiplos processos leem e escrevem dados compartilhados e o resultado depende de quem executa primeiro.

Race Condition - Características do Problema

- Execução concorrente (threads ou processos)
- Compartilhamento de memória
- Ausência de sincronização adequada
- Resultados inconsistentes ou incorretos

➢ Não é erro de lógica

➢ É erro de controle de concorrência

Exemplo Clássico (Incremento)

Código conceitual:

```
int counter = 0;

Thread A: counter = counter + 1;
Thread B: counter = counter + 1;
```

Esperado:

```
counter = 2
```

Possível:

```
counter = 1
```

O que realmente acontece?

Decomposição da operação:

```
counter = counter + 1;
```

Equivale a:

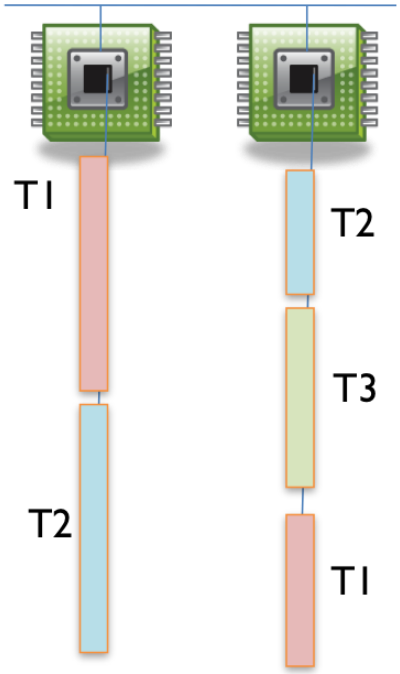
1. Ler conter
2. Somar 1
3. Escrever resultado

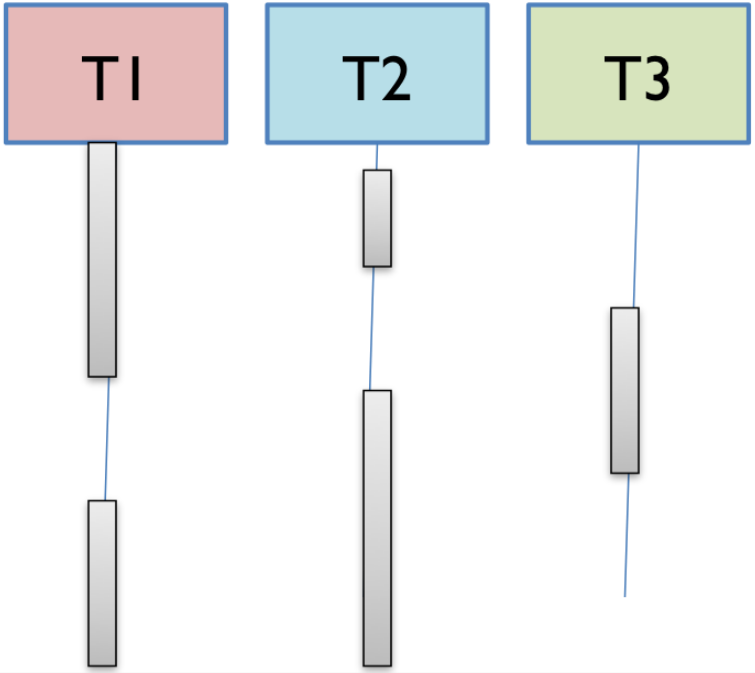
Interleaving (Execução intercalada)

Passo	Thread A	Thread B
1	lê 0	
2		lê 0
3	soma → 1	
4		soma → 1
5	escreve 1	
6		escreve 1

Resultado final: 1 (ERRO!)

Interleaving (Execução intercalada)





Região Crítica

Conceito central (Tanenbaum):

- Parte do código que acessa dados compartilhados

```
counter = counter + 1; // Região Crítica
```

Problema:

- Múltiplas threads entram ao mesmo tempo

Condições para Solução Correta

Uma solução deve garantir:

- Exclusão mútua
- Progresso
- Espera limitada (bounded waiting)
- Independência de velocidade/processadores

Exclusão Mútua

Garantir que apenas uma thread execute a região crítica por vez

- Evita inconsistência
- Garante integridade dos dados

O que é: Uma propriedade de sincronização que garante que apenas uma thread por vez entre na região crítica (o trecho de código que manipula o dado compartilhado).

Como funciona: Antes de acessar um recurso compartilhado, uma thread deve "bloquear" (lock) um Mutex. Se outra thread tentar bloquear o mesmo Mutex, ela será colocada em espera até que a primeira thread "desbloqueie" (unlock) o recurso.

Solução com Mutex

Pseudocódigo em C:

```
pthread_mutex_lock(&lock);
counter = counter + 1;
pthread_mutex_unlock(&lock);
```

- Apenas uma thread entra por vez
- Serializa acesso à região crítica

Exemplo de Race Condition em C:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int contador = 0;

void* incrementar(void* arg) {
    for (int i = 0; i < 100000; i++) {
        contador++;
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, incrementar, NULL);
    pthread_create(&t2, NULL, incrementar, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Valor final do contador: %d\n", contador);
    return 0;
}
```

Exemplo de Race Condition com MUTEX em C:

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int contador = 0;
pthread_mutex_t mutex;

void* incrementar(void* arg) {
    for (int i = 0; i < 1000; i++) {
        pthread_mutex_lock(&mutex);
        contador++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&t1, NULL, incrementar, NULL);
    pthread_create(&t2, NULL, incrementar, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&mutex);

    printf("Valor final do contador: %d\n", contador);
    return 0;
}
```

Hora de perguntar, dúvidas

Próxima aula

- Fundamentos e Algoritmos de Escalonamento: objetivos (tempo de resposta, throughput, utilização) e algoritmos clássicos — FCFS, SJF, Round Robin e Prioridades.
- Análise prática de desempenho: simulação de escalonamento com cálculo de tempo de espera e resposta comparando diferentes algoritmos.
- Concorrência e Deadlock: sincronização (exclusão mútua, semáforos, locks) e estudo de deadlock (condições, identificação e estratégias de prevenção/detecção/recuperação).